

Vibe Coding a Real SaaS: The 15-Layer Playbook

How to get agentic coders like Claude Code or OpenAI Codex to handle everything beyond "AI prompt → website built."

How to use this document

A prompt can get you a website. A SaaS needs fifteen more layers underneath it. This playbook takes each layer and gives you four things:

1. **What it is**, in plain terms.
2. **What AI agents get wrong** when you don't ask explicitly. Agents optimise for "it runs on my machine." These are the blind spots.
3. **Decisions you must make yourself**. The agent can recommend, but you own these choices.
4. **Copy-paste prompts** for Claude Code or Codex, plus a **verification checklist** so you can check the agent's work instead of trusting it.

The layers are ordered in the sequence you should actually build them, not the order in the reel.

#	Layer	Build phase
1	System design	Plan
2	System architecture	Plan
3	Databases & storage	Foundation
4	Auth & permissions	Foundation
5	APIs & backend logic	Foundation
6	Frontend	Product
7	CI/CD & version control	Ship
8	Testing	Ship
9	Hosting & cloud	Ship
10	Security	Harden
11	Rate limiting	Harden
12	Caching & CDN	Harden
13	Error tracking & logs	Operate
14	Monitoring & alerts	Operate
15	Scaling	Grow

Part 0: Set up the agent before layer one

Most bad AI-built SaaS apps fail here, before a single feature is written. An agent with no project memory makes fresh, inconsistent decisions every session.

0.1 Create a project memory file

- **Claude Code** reads a `CLAUDE.md` file in your repo root at the start of every session.
- **Codex** reads an `AGENTS.md` file the same way.

You can keep one file and symlink or copy it so both agents share the same rules. Start it like this:

```
# Project: [Name]

## What this product does
[2-3 sentences. Who the user is, what problem it solves.]

## Stack (do not change without asking)
- Frontend: [e.g. Next.js + TypeScript + Tailwind]
- Backend: [e.g. Next.js API routes / Node + Fastify / Python + FastAPI]
- Database: [e.g. Postgres via Supabase / Neon], ORM: [e.g. Prisma / Drizzle]
- Auth: [e.g. Clerk / Supabase Auth / Auth.js]
- Hosting: [e.g. Vercel + Railway]

## Rules
- Never commit secrets. All secrets go in environment variables, documented in .env.example.
- Every new API endpoint needs: input validation, auth check, authorization check, tests.
- Every database change goes through a migration file. Never edit the DB by hand.
- Run lint, typecheck and tests before saying a task is done.
- Ask before adding a new dependency.
- Prefer small, reviewable changes. One feature per branch.

## Commands
- Dev: [npm run dev]
- Test: [npm test]
- Lint/typecheck: [npm run lint && npm run typecheck]
- Migrate: [npx prisma migrate dev]

## Architecture decisions
See /docs/decisions/ for the log of decisions and why they were made.
```

Update this file every time you make a decision. It's the single highest-leverage habit in agentic coding.

0.2 The working loop

For every layer below, use this loop:

1. **Plan first, code second.** In Claude Code, use plan mode (so it proposes before editing). In Codex, ask for a plan and tell it not to write code yet.
2. **Review the plan.** This is where you catch bad decisions cheaply.
3. **Let it implement in small steps**, on a branch.
4. **Make it verify its own work:** run tests, lint, typecheck, and show you the output.

5. **Review the diff yourself** before merging.
6. **Record the decision** in `CLAUDE.md` / `AGENTS.md` or `/docs/decisions/`.

0.3 Useful agent features to lean on

- **Custom slash commands / reusable prompts.** Save prompts from this document as files (in Claude Code, `.claude/commands/`) so you can run `/security-review` instead of pasting.
- **Subagents.** Claude Code can delegate to specialised subagents (for example, a "reviewer" that only critiques). Useful for security and test reviews.
- **Hooks.** Claude Code can run commands automatically on events, e.g. run the linter after every file edit.
- **MCP servers.** Connect the agent to real tools (your database, GitHub, error tracker, docs) so it works from real data instead of guessing.

Features change quickly. Check the current docs: Claude Code at <https://docs.claude.com/en/docs/claude-code/overview> and Codex at OpenAI's developer docs.

0.4 The universal "don't fool me" prompt

Append this to any big task:

Before you say this is done:

1. Run lint, typecheck and the full test suite, and show me the output.
2. List every file you changed and why.
3. List anything you skipped, stubbed, mocked or hard-coded.
4. List assumptions you made that I should confirm.

Do not describe work as complete if any test is failing or skipped.

1. System design

What it is

The "what and how much" of your product before any code: who the users are, what they do, how much data and traffic you expect, what must never break, and what trade-offs you're making.

What AI agents get wrong

- Jump straight to code and invent requirements you never agreed to.
- Design for a toy (one user, no concurrency) or wildly over-engineer (microservices for 10 users).
- Ignore multi-tenancy, i.e. how one customer's data stays separate from another's.

Decisions you own

- Who the customer is: individuals, teams, or companies (this decides your whole data model).
- Pricing model: free tier, per seat, usage-based.
- Realistic load for year one: users, requests per day, data size.
- What's non-negotiable: uptime, data privacy, compliance (GDPR, DPDP Act in India, HIPAA, etc.).

Prompts

1a. Requirements interview (use before anything else)

I'm building a SaaS: [one paragraph description].

Don't write any code. Act as a senior product engineer and interview me.

Ask me questions, one group at a time, about:

- users and roles
- core workflows (the 3–5 things users actually do)
- tenancy (individual vs team vs organisation accounts)
- data we store and how sensitive it is
- expected scale in year one
- billing and plans
- compliance or regional requirements
- integrations with other services

After I answer, write /docs/system-design.md containing: problem statement, user roles, core user flows, functional requirements, non-functional requirements (performance, availability, security), out-of-scope items, and open questions.

1b. Stress-test the design

Read /docs/system-design.md. Act as a skeptical staff engineer reviewing it. Identify: missing requirements, ambiguous flows, scaling risks, security and privacy risks, and anything over-engineered for our stage. Rank issues by severity. Suggest the simplest design that meets the requirements. Don't edit the file; give me the review as a list.

1c. Capacity back-of-envelope

Based on `/docs/system-design.md`, estimate for year one: requests per second at peak, database size, storage size for uploaded files, and background job volume. Show your math. Tell me which component becomes the bottleneck first.

Verify

- ☐ `/docs/system-design.md` exists and you've read every line.
 - ☐ Tenancy model is written down explicitly.
 - ☐ There's an "out of scope" section. If there isn't, the agent will keep adding features.
-

2. System architecture

What it is

The concrete building blocks: which services exist, how they talk, where data lives, what runs in the background, and which third-party services you rely on.

What AI agents get wrong

- Pick whatever stack is most common in training data, not what fits you.
- Add dependencies freely, so you end up with three date libraries and two HTTP clients.
- Mix concerns: business logic in UI components, database calls in route handlers everywhere.
- Forget background jobs (emails, webhooks, exports) and do everything inside a request.

Decisions you own

- Monolith vs services. **For almost every early SaaS: a modular monolith.**
- Managed services vs self-hosted.
- Language and framework, based on what *you* can debug at 2 a.m.

Prompts

2a. Architecture proposal

Read `/docs/system-design.md`. Propose an architecture for this SaaS at our current stage. Optimise for: a solo/small team, low ops burden, and easy debugging. Prefer a modular monolith unless there's a strong reason not to.

Give me:

1. Component diagram (as a Mermaid diagram)
2. Stack choice for each layer, with one alternative and why you rejected it
3. Folder structure for the repo
4. Where background jobs run and how
5. List of third-party services, with cost at our scale
6. The 3 riskiest decisions in this architecture

Write it to `/docs/architecture.md`. Don't create any code yet.

2b. Decision records

For each major decision in `/docs/architecture.md`, create an Architecture Decision Record in `/docs/decisions/` named `NNNN-short-title.md`, with: Context, Decision, Alternatives considered, Consequences. Then add a summary of the stack and folder structure to `CLAUDE.md` (and `AGENTS.md`) under "Stack" and "Architecture".

2c. Scaffold

Scaffold the project exactly as described in `/docs/architecture.md`.
Include: folder structure, TypeScript strict mode (or equivalent), linter, formatter, `.env.example` with every variable documented, a README with setup steps, and a health-check endpoint at `/api/health`.
Don't build features. Run the dev server and the health check to prove it works.

Verify

- ☐ You can explain every box in the architecture diagram.
 - ☐ Every dependency in `package.json` (or equivalent) has a reason.
 - ☐ Business logic lives in a dedicated layer (e.g. `/services` or `/domain`), not inside UI or route files.
-

3. Databases & storage

What it is

Where your data lives: a relational database for structured data, object storage (S3, R2, Supabase Storage) for files, and sometimes a cache or queue.

What AI agents get wrong

- No migrations; they edit schemas directly or regenerate them from scratch.
- Missing indexes, so things are fine with 100 rows and slow at 100,000.
- No `tenant_id` / `organization_id` on tables, which causes data leaks between customers.
- Storing files in the database, or making uploaded files publicly readable.
- No backups, no soft deletes, no `created_at` / `updated_at`.

Decisions you own

- Database: Postgres is the safe default for almost any SaaS.
- Hosted provider (Supabase, Neon, RDS, PlanetScale, etc.).
- Backup and retention policy.
- What gets deleted when a user deletes their account.

Prompts

3a. Schema design

Based on `/docs/system-design.md`, design the database schema.

Requirements:

- Multi-tenant: every tenant-owned table has `organization_id`, with a foreign key
- Every table has `id` (UUID), `created_at`, `updated_at`
- Use soft deletes (`deleted_at`) where users might want to recover data
- Add indexes for every foreign key and every column we filter or sort by
- Use proper constraints (NOT NULL, UNIQUE, CHECK) instead of relying on app code
- Money stored as integer minor units (paise/cents), never floats

Output: an ERD as a Mermaid diagram, then the schema in our ORM format, then an explanation of each index. Don't run migrations yet.

3b. Migrations and seed data

Create the initial migration from the approved schema. Then write a seed script that creates 2 organisations, each with 3 users and realistic sample data, so I can test that tenants can't see each other's data. Run the migration and seed against the local database and show me the output.

3c. File storage

Implement file uploads using [S3 / R2 / Supabase Storage].

- Files are private by default
 - Upload via pre-signed URLs directly from the browser (don't stream through our server)
 - Validate file type and size on the server before issuing the URL
 - Store only the file key and metadata in the database
 - Downloads use short-lived signed URLs, and only after an authorization check
 - Scope storage paths by `organization_id`
- Write tests for: wrong file type, too large, and user from another org trying to download.

3d. Performance review

Review every database query in the codebase. Flag:

- N+1 queries
 - queries without a supporting index
 - `SELECT *` where we only need a few columns
 - missing pagination on list endpoints
 - any query that doesn't filter by `organization_id` on a tenant table
- Give me a table: file, line, problem, fix. Then fix the high-severity ones.

3e. Backups

Document our backup setup in `/docs/runbooks/backups.md`: how often backups run, how long they're kept, and step-by-step how to restore to a point in time. Then write a script that restores the latest backup into a separate database so we can test restores without touching production.

Verify

- ☐ Every schema change is a migration file in version control.
 - ☐ Log in as a user from Org A and try to fetch Org B's data by changing an ID in the URL. It must fail.
 - ☐ You have actually restored a backup at least once.
-

4. Auth & permissions

What it is

Authentication = who are you (login, signup, sessions, password reset, SSO). **Authorization** = what are you allowed to do (roles, permissions, which organisation's data you can see).

What AI agents get wrong

- Hand-rolling password hashing and session logic. **Use a proven provider or library.**
- Checking permissions only in the frontend (hiding a button) instead of on the server.
- Checking that a user is logged in, but not that they own the resource they're requesting. This is the single most common SaaS vulnerability (called IDOR or broken object-level authorization).
- Forgetting email verification, password reset expiry, and session revocation.

Decisions you own

- Auth provider: Clerk, Auth0, Supabase Auth, Auth.js, WorkOS (if you need enterprise SSO).
- Roles: start simple, e.g. Owner, Admin, Member, Viewer.
- Login methods: email + password, magic link, Google, SSO.

Prompts

4a. Authentication

```
Implement authentication using [provider]. Do not hand-roll password hashing
or session management.
Include: signup, login, logout, email verification, password reset,
Google login, and session expiry. Protect all routes under /app and /api
except the ones I list: [public routes].
Add a middleware that attaches the current user and current organization
to every request.
```

4b. Authorization model

```
Design a role-based permission system for these roles: [Owner, Admin, Member, Viewer].
1. Write a permissions matrix (role × action) as a table in /docs/permissions.md.
2. Implement a single central function, e.g. can(user, action, resource),
   that every endpoint uses. No ad-hoc role checks scattered around.
3. Every read or write of a tenant resource must verify the resource's
   organization_id matches the user's current organization.
4. Frontend hides actions the user can't take, but the server is the
   source of truth.
```

4c. Authorization tests (critical)

Write integration tests that prove, for every API endpoint:

- unauthenticated requests get 401
- authenticated users without permission get 403
- a user from Org A gets 404 (not 403) when requesting Org B's resource by ID
- each role can do exactly what /docs/permissions.md says, nothing more

Generate these tests from the permissions matrix so they stay in sync.

4d. Team features

Implement: invite a teammate by email, accept invite, change role, remove member, transfer ownership, and leave organization.
Edge cases to handle: last owner can't leave, invite links expire after 7 days, removed members lose access immediately (revoke their sessions).

Verify

- ☐ Open DevTools, copy an API request, change the resource ID to one from another org. You get 404.
 - ☐ A Viewer calling a delete endpoint directly (not via UI) gets 403.
 - ☐ No file contains custom password hashing code.
-

5. APIs & backend logic

What it is

The server-side rules of your product: endpoints, validation, business logic, background jobs, webhooks, and integrations like payments.

What AI agents get wrong

- No input validation; trusting whatever the client sends.
- Inconsistent error formats across endpoints.
- Doing slow work (sending emails, calling other APIs) inside the request.
- Non-idempotent payment and webhook handling, which leads to double charges or duplicate records.
- No pagination on list endpoints.

Decisions you own

- REST vs tRPC vs GraphQL (REST or tRPC is plenty for most).
- Payment provider: Stripe, Razorpay (India), Paddle/Lemon Squeezy (merchant of record handles tax).
- Background job system: Inngest, Trigger.dev, BullMQ, Cloud Tasks, etc.

Prompts

5a. API conventions

Define our API conventions and write them to `/docs/api-conventions.md`, then add a summary to `CLAUDE.md`. Include:

- URL and naming style
- a single error response format with error codes
- validation with [Zod / Pydantic] on every input, at the boundary
- pagination format (cursor-based) for all list endpoints
- how auth and org context are read in every handler
- HTTP status codes we use and when

Then refactor existing endpoints to match.

5b. Build a feature endpoint

Build the API for [feature, e.g. "projects": create, list, get, update, delete]. Follow `/docs/api-conventions.md` and `/docs/permissions.md`.

- Business logic goes in `/services`, route handlers stay thin
- Validate every input
- Authorization check on every operation via `can()`
- Filter every query by `organization_id`
- Cursor pagination on list

Write unit tests for the service and integration tests for each endpoint, including permission and cross-tenant cases.

5c. Background jobs

Set up background jobs using [tool]. Move these out of the request path: [sending emails, generating exports, calling third-party APIs]. Each job must: be idempotent (safe to run twice), retry with exponential backoff, have a max retry count, and log failures with enough context to debug. Add a way to see failed jobs.

5d. Payments and webhooks

Integrate [Stripe / Razorpay] subscriptions for these plans: [plans].

- Treat webhooks as the source of truth for subscription state, not the redirect after checkout
- Verify webhook signatures
- Make webhook handlers idempotent by storing processed event IDs
- Handle: payment success, failure, cancellation, plan change, refund
- Gate features by plan with a single helper, e.g. `hasFeature(org, feature)`

Write tests that replay each webhook event twice and assert no duplicates. Use test mode keys only.

Verify

- ☐ Send garbage JSON to an endpoint. You get a clean 400 with your standard error format, not a stack trace.
 - ☐ Replay the same webhook twice. Nothing duplicates.
 - ☐ Any request that takes more than ~1 second has a reason, or has been moved to a background job.
-

6. Frontend

What it is

Everything the user sees and touches: pages, forms, loading states, errors, responsiveness, accessibility.

What AI agents get wrong

- Only build the "happy path". No loading, empty, or error states.
- Generic, templated look that screams "AI made this".
- Poor accessibility: missing labels, no keyboard navigation, low contrast.
- Giant components, duplicated UI code, inconsistent spacing.
- Leaking secrets into client-side code.

Decisions you own

- Design direction and brand. Give the agent references and screenshots.
- Component library (shadcn/ui, Radix, MUI, etc.).
- Which screens actually matter for launch.

Prompts

6a. Design system first

Before building screens, create a small design system:

- color tokens (with dark mode), typography scale, spacing scale, radii
- base components: Button, Input, Select, Modal, Toast, Table, EmptyState, ErrorState, Skeleton
- a /design page that shows every component in every state

Style direction: [describe, or reference 2–3 products you like].
Avoid generic template look. Use these components everywhere from now on.

6b. Build a screen properly

Build the [screen name] screen using our design system and the existing API.
Every data-driven view must handle 4 states: loading (skeletons), empty (helpful message + primary action), error (message + retry), and success. Forms need: client-side validation matching the server rules, disabled submit while pending, inline field errors, and success feedback.
Must work at 375px mobile width and on desktop.

6c. Accessibility pass

Audit the frontend for accessibility (WCAG 2.1 AA): labels on all inputs, keyboard navigation for every interactive element, visible focus states, colour contrast, alt text, correct heading order, and ARIA only where needed.
Fix issues and add an automated accessibility check (e.g. axe) to our tests.

6d. Frontend secrets check

Search the frontend bundle and all client-side code for API keys, secrets or private URLs. Only variables intended to be public should be exposed to the browser. Report what you found and fix it.

Verify

- ☐ Turn off your Wi-Fi and use the app. Every screen shows a sensible error.
 - ☐ Use the whole app with only a keyboard.
 - ☐ Open it on your actual phone.
-

7. CI/CD & version control

What it is

Version control = Git: every change tracked, reviewable, reversible. **CI** (Continuous Integration) = every push automatically runs lint, typecheck, tests. **CD** (Continuous Deployment) = passing code deploys automatically to staging/production.

What AI agents get wrong

- Huge commits mixing ten unrelated changes.
- Committing `.env` files or secrets.
- No CI, so broken code reaches production.
- Running database migrations manually and inconsistently.

Decisions you own

- Branching: `main` is always deployable; feature branches + pull requests.
- Environments: at minimum local, staging (preview), production.
- Whether production deploys are automatic or need your approval.

Prompts

7a. Git hygiene

Set up version control hygiene:

- `.gitignore` that covers env files, build output, OS files, editor files
- pre-commit hook that runs lint and format on staged files
- a secret scanner in pre-commit (e.g. gitleaks)
- commit message convention (Conventional Commits) documented in `CLAUDE.md`

Then scan the full git history for any committed secrets and tell me what you find.

7b. CI pipeline

Create a GitHub Actions CI workflow that runs on every pull request and push to main:

install (with caching) → lint → typecheck → unit tests → integration tests against a real Postgres service container → build.

Fail fast. Keep it under 10 minutes. Add a status badge to the README.

7c. CD pipeline

Set up deployments:

- every PR gets a preview deployment
- merge to main deploys to staging automatically
- production deploy requires a manual approval step
- database migrations run automatically as part of deploy, before the new code goes live, and the deploy fails if a migration fails
- document how to roll back to the previous version in `/docs/runbooks/rollback.md`

7d. Working with the agent in Git

From now on: create a new branch for each task, make small commits with clear messages, and open a pull request with a description of what changed, why, how to test it, and any risks. Never push directly to main.

Verify

- ☐ Open a PR with a deliberately failing test. CI blocks it.
 - ☐ You have rolled back a deployment once, on purpose, to practise.
 - ☐ `git log` reads like a story, not "fix", "fix2", "final fix".
-

8. Testing

What it is

Automated checks that prove your app does what it should, and keeps doing it when you (or the agent) change things. Types: unit (one function), integration (API + database), end-to-end (a real browser clicking through).

What AI agents get wrong

- Write tests that test the mocks, not the real behaviour.
- **Change the test to make it pass** instead of fixing the code. Watch for this constantly.
- Skip or delete failing tests quietly.
- Only test happy paths.

Decisions you own

- What must never break: signup, login, payments, core workflow, data isolation. These get end-to-end tests.
- Coverage target (a sensible range is 70–80% on business logic; 100% is rarely worth it).

Prompts

8a. Test strategy

```
Write /docs/testing.md with our test strategy:  
- unit tests for /services business logic  
- integration tests for every API endpoint using a real test database  
- end-to-end tests (Playwright) for these critical flows: signup, login,  
  [core workflow], upgrade plan, invite teammate  
- which things are mocked (external APIs only) and which are never mocked  
  (our database, our auth checks)  
Set up the tooling for all three levels.
```

8b. Tests for existing code

```
Write tests for [module]. For each function, cover: happy path, invalid  
input, boundary values, permission denied, and cross-tenant access.  
Rules: do not modify the code under test. If a test reveals a bug, stop  
and report it to me rather than changing the test to pass.
```

8c. Test-driven bug fixing

```
Bug: [describe the bug and how to reproduce].  
1. First write a failing test that reproduces it. Show me it failing.  
2. Then fix the code.  
3. Show me the test passing and the full suite still green.
```

8d. Test quality review

Review our test suite critically. Find: tests that would still pass if the feature were broken, tests that only assert on mocks, skipped tests, flaky tests, and important paths with no coverage. Give me a prioritised list.

Verify

- ☐ Deliberately break a line of business logic. At least one test fails.
 - ☐ `grep` for `.skip`, `xit`, `@pytest.mark.skip`. Every skip has a reason.
 - ☐ After each agent session, check the diff for changes to test files you didn't ask for.
-

9. Hosting & cloud

What it is

Where your app runs in production: frontend hosting, backend servers, the database, file storage, DNS, domains, SSL, and environment variables.

What AI agents get wrong

- Suggest complex setups (Kubernetes, Terraform everything) way too early.
- Use the same database for staging and production.
- Hard-code URLs and config that differ between environments.
- Forget about cost, so you get a surprise bill.

Decisions you own

- Platform: Vercel, Netlify, Railway, Render, Fly.io, or AWS/GCP/Azure. Managed platforms are usually the right early choice.
- Region: close to your users (and for data residency rules).
- Budget ceiling and billing alerts.

Prompts

9a. Hosting plan

```
Recommend a hosting setup for our architecture in /docs/architecture.md.  
Constraints: small team, minimal ops, users mostly in [region],  
budget around [amount]/month at launch.  
For each component (frontend, backend, database, storage, background jobs,  
email), give provider, plan, estimated monthly cost at launch and at 10x  
users, and what we'd need to change at 10x.  
Write it to /docs/hosting.md.
```

9b. Environment configuration

```
Make the app fully configurable by environment variables. Produce:  
- .env.example listing every variable with a comment explaining it  
- runtime validation at startup that fails loudly if a required  
  variable is missing or malformed  
- separate values for local, staging, production  
- no URLs, keys or IDs hard-coded anywhere  
List every place you changed.
```

9c. Production readiness

Prepare the app for production deployment on [platform]:

- custom domain and HTTPS
 - security headers
 - separate production database with automated backups enabled
 - health check endpoint wired to the platform
 - graceful shutdown so in-flight requests finish during deploys
- Then write `/docs/runbooks/deploy.md` with step-by-step first-deploy instructions.

Verify

- ☐ Staging and production use different databases and different API keys.
 - ☐ Billing alerts are set on every provider.
 - ☐ Removing a required env variable makes the app refuse to start, with a clear message.
-

10. Security

What it is

Protecting your users' data and your system from attacks: injection, cross-site scripting, broken access control, leaked secrets, vulnerable dependencies, and more.

What AI agents get wrong

- Build what you ask without thinking like an attacker.
- Trust user input in queries, HTML, file paths and redirects.
- Log sensitive data (passwords, tokens, personal info).
- Add outdated or vulnerable packages.
- Leave debug endpoints and verbose errors enabled in production.

Decisions you own

- What data is sensitive and how it's protected.
- Compliance obligations for your market.
- Whether to get an external penetration test before handling sensitive data.

Prompts

10a. Threat model

```
Create a threat model for our app in /docs/security/threat-model.md.  
List our assets (data, accounts, payments), entry points (every endpoint,  
upload, webhook, form), and for each entry point the relevant threats from  
the OWASP Top 10. Rate likelihood and impact, and list mitigations we have  
and mitigations we're missing.
```

10b. Security review (run before every release)

```
Act as a security reviewer. Audit the codebase for:  
- broken access control and missing org scoping  
- SQL/NoSQL injection and unsafe raw queries  
- XSS (unsafe HTML rendering)  
- CSRF on state-changing requests  
- open redirects and SSRF (server fetching user-supplied URLs)  
- insecure file uploads  
- secrets in code or logs, sensitive data in logs  
- missing security headers (CSP, HSTS, X-Frame-Options, etc.)  
- verbose error messages leaking internals  
- insecure cookie settings  
Report each finding with file, line, severity, exploit scenario and fix.  
Do not fix anything yet.
```

10c. Dependency security

Run a dependency vulnerability audit. List vulnerable packages, severity, and whether we actually use the vulnerable code path. Upgrade what's safe, and explain anything you can't upgrade. Then set up automated dependency update PRs (e.g. Dependabot or Renovate) and add the audit to CI.

10d. Data protection

Implement: encryption of sensitive fields at rest [list fields], redaction of passwords, tokens, and personal data from all logs, account deletion that removes or anonymises the user's data, and a data export feature so users can download their data.

Verify

- ☐ Run the review prompt with a fresh session or a separate reviewer subagent, not the one that wrote the code.
- ☐ Try pasting `<script>alert(1)</script>` into every text field. Nothing pops up.
- ☐ Check your logs for any email addresses, tokens or passwords.

AI security reviews catch a lot, but they're not a substitute for a professional audit if you handle payments, health, financial or other sensitive data.

11. Rate limiting

What it is

Capping how many requests someone can make in a time window. It protects you from brute-force logins, scrapers, abuse, runaway scripts and surprise bills (especially if you call paid AI APIs).

What AI agents get wrong

- Don't add it at all unless asked.
- Use in-memory counters that reset on deploy and don't work across multiple servers.
- Apply one global limit instead of different limits per route and per plan.

Decisions you own

- Limits per route: login and signup strictest, expensive endpoints next, reads loosest.
- Limits per plan (free vs paid).
- What happens when a limit is hit: block, queue, or charge overage.

Prompts

11a. Rate limiting setup

```
Implement rate limiting using a shared store (e.g. Redis / Upstash) so it
works across multiple server instances.
Limits:
- login, signup, password reset: [5 per 15 min per IP and per email]
- expensive endpoints [list, e.g. AI generation, exports]: [N per minute per org]
- general API: [N per minute per user]
- public/unauthenticated endpoints: [N per minute per IP]
Return 429 with a Retry-After header and our standard error format.
Make limits configurable per plan.
Write tests that exceed each limit.
```

11b. Cost protection for AI or paid APIs

```
We call [paid API, e.g. an LLM API]. Add:
- per-org daily and monthly usage quotas tied to plan
- usage tracking stored in the database
- a hard global spending cap that disables the feature if exceeded
- an alert to me when any org hits 80% of quota or the global cap nears
- a usage page in the UI so customers see their consumption
```

11c. Abuse review

```
Review our public surface for abuse risks: signup spam, email sending
abuse (invites, password resets), enumeration of users via error messages,
scraping of public pages, and webhook endpoints that could be flooded.
Propose protections for each, including bot protection on signup.
```

Verify

- ☐ Write a quick loop that hits login 20 times. It gets blocked.
 - ☐ Limits still work after a redeploy.
 - ☐ "Wrong password" and "no such user" return the same message.
-

12. Caching & CDN

What it is

Caching = storing results so you don't recompute or refetch them every time. **CDN** (Content Delivery Network) = serving static files (JS, CSS, images) from servers close to the user.

What AI agents get wrong

- Cache too aggressively, so users see stale or, worse, **another tenant's** data.
- Cache nothing, so every page hits the database.
- Forget cache invalidation when data changes.
- Serve unoptimised, huge images.

Decisions you own

- What can be stale and for how long.
- Cache store (often your hosting platform's CDN plus Redis for app data).

Prompts

12a. Caching strategy

Analyse our app and propose a caching strategy in /docs/caching.md.
For each candidate (static assets, public marketing pages, API responses, expensive queries, third-party API results), specify: where it's cached (browser, CDN, server, Redis), TTL, cache key, and how it's invalidated when data changes.
Rule: any cache of tenant data must include `organization_id` in the key, and authenticated responses must never be cached at a shared CDN layer.

12b. Implement CDN and static assets

Configure static assets to be served via CDN with long cache lifetimes and content-hashed filenames. Optimise images (modern formats, responsive sizes, lazy loading). Set correct Cache-Control headers for: static assets, public pages, and authenticated API responses (private, no-store where needed). Show me the headers for one example of each.

12c. Application caching

Add Redis caching for these slow operations: [list]. Use a cache-aside pattern with keys that include `organization_id`. Invalidate on every write that affects the data. Add a way to bypass the cache for debugging. Write tests proving that: cache is invalidated on update, and two different orgs never receive each other's cached data.

Verify

- ☐ Log in as two users from different orgs in two browsers. Neither ever sees the other's data, even after refreshes.
 - ☐ Update a record. The new value shows up immediately (or within the TTL you agreed to).
 - ☐ Run a Lighthouse audit on your main pages.
-

13. Error tracking & logs

What it is

Error tracking captures every crash with the stack trace, user and context (e.g. Sentry). **Logs** are a record of what your system did, searchable when something goes wrong.

What AI agents get wrong

- `console.log` everywhere, with no structure and no levels.
- Swallowing errors with empty `catch` blocks so failures are silent.
- Logging sensitive data.
- No way to trace one user's request through the system.

Decisions you own

- Error tracking tool (Sentry, Rollbar, Highlight, etc.).
- Log destination and retention (your platform's logs, Axiom, Better Stack, Datadog, etc.).

Prompts

13a. Structured logging

Replace all `console.log/print` calls with a structured logger (JSON output, levels: debug, info, warn, error).

- Every request gets a request ID, included in every log line and returned in a response header
- Every log line includes `user_id` and `organization_id` when available
- Automatically redact fields named `password`, `token`, `secret`, `authorization`, and any fields in [list of personal data]
- debug level is off in production

Show me an example log line for a request.

13b. Error tracking

Integrate [Sentry] on both frontend and backend.

- Attach `user_id`, `organization_id` and request ID to every error (no emails or personal data unless I approve)
- Upload source maps so stack traces are readable, but don't serve them publicly
- Separate environments (staging vs production)
- Add a test route, only in staging, that throws an error so I can verify it arrives

13c. Error handling cleanup

Find every empty or overly broad catch block, every ignored promise rejection, and every place an error is caught but not logged or re-thrown. Fix them so errors are either handled meaningfully or reported. Make sure users see a friendly message while the full detail goes to error tracking.

Verify

- ☐ Trigger an error in staging. It appears in your error tracker with a readable stack trace within a minute.
 - ☐ Take a request ID from a response header and find every log line for that request.
 - ☐ Search your logs for "password". Zero results.
-

14. Monitoring & alerts

What it is

Knowing your app is healthy **before your users tell you it isn't**: uptime checks, performance metrics, business metrics, and alerts that wake you up only when it matters.

What AI agents get wrong

- Nothing is set up, because nothing is visible until it breaks.
- Too many noisy alerts, so you start ignoring all of them.
- Monitoring only the server, not whether users can actually do the core thing.

Decisions you own

- What counts as "down" or "degraded" for your product.
- Who gets alerted, how (email, Slack, SMS, phone call) and when.
- Your service level target (e.g. 99.9% uptime).

Prompts

14a. Monitoring plan

```
Write /docs/monitoring.md defining:  
- uptime checks for: homepage, login, /api/health, and [core workflow endpoint]  
- key technical metrics: error rate, p95 latency per endpoint, database  
  connections, background job queue depth and failure rate  
- key business metrics: signups, active orgs, successful payments, failed payments  
- alert rules: what threshold, how long, severity, and who is notified  
Keep alerts to only things that need a human to act.
```

14b. Health checks

```
Upgrade /api/health to a real health check: verify database connectivity,  
cache connectivity, and background job system, with timeouts, returning  
component-level status. Keep a lightweight /api/health/live for the load  
balancer that only confirms the process is up.
```

14c. Alerts and runbooks

```
For each alert in /docs/monitoring.md, write a runbook in /docs/runbooks/  
with: what the alert means, how to check impact, likely causes, step-by-step  
fixes, and when to escalate. Link each alert to its runbook.
```

14d. Status and incident process

```
Set up a simple public status page and write /docs/runbooks/incident.md:  
how to declare an incident, how to communicate with customers, and a  
template for a short post-incident review.
```

Verify

- ☐ Stop your database in staging. An alert reaches you within the time you defined.
 - ☐ Every alert has a runbook.
 - ☐ You've looked at your dashboard at least once a week.
-

15. Scaling

What it is

Handling more users, data and traffic without slowing down or falling over, and without costs exploding.

What AI agents get wrong

- Over-engineer for scale you don't have (sharding, microservices, Kubernetes on day one).
- Or ignore the basics that actually break first: missing indexes, N+1 queries, no connection pooling, synchronous heavy work.

Decisions you own

- When to scale. **Measure first, then optimise the actual bottleneck.**
- Budget for infrastructure as you grow.

Prompts

15a. Load test

Write load tests (e.g. k6) for our 5 most important endpoints and the core user flow. Simulate [N] concurrent users ramping up over 10 minutes against staging. Report p50/p95/p99 latency, error rate, and the point where performance degrades. Identify the bottleneck component.

15b. Scaling readiness review

Review the app for scaling readiness:

- is the app stateless (no in-memory sessions, uploads, or rate limit counters) so we can run multiple instances?
- database connection pooling configured correctly for serverless / multiple instances?
- slow queries and missing indexes (use EXPLAIN on the top queries)
- heavy work that should be in background jobs
- unbounded queries or list endpoints without pagination
- large payloads

Rank by what will break first as traffic grows 10x and 100x.

15c. Targeted optimisation

Our bottleneck is [X, from load test or monitoring]. Propose 3 options to fix it, from simplest to most complex, with cost and effort for each. Implement the simplest one that meets a target of [p95 < 300ms at N users]. Re-run the load test and show before/after numbers.

15d. Cost at scale

Using `/docs/hosting.md` and our current usage, model monthly infrastructure cost at 10x and 100x users. Identify the most expensive component at each level and suggest ways to reduce it.

Verify

- ☐ You have a load test result with real numbers, not a guess.
 - ☐ You can run two instances of your backend at once without anything breaking.
 - ☐ Every optimisation has before/after numbers.
-

16. ...and more

The list never really ends. Once the fifteen layers are solid, these come next:

- **Emails:** transactional email provider, templates, SPF/DKIM/DMARC so you don't land in spam.
- **Analytics:** product analytics to see what users actually do.
- **Feature flags:** release to a few users first, turn off instantly if it breaks.
- **Legal:** terms, privacy policy, cookie consent, data processing agreements.
- **Admin panel:** internal tools to support customers safely, with audit logs.
- **Audit logs:** record of who did what in each organisation (enterprise buyers ask for this).
- **Onboarding:** first-run experience, sample data, docs, help center.
- **Internationalisation:** languages, currencies, time zones.
- **Disaster recovery:** a tested plan for "our main provider is down" or "data was deleted".

Use the same pattern for each: ask the agent to plan, review the plan, implement small, verify, and record the decision.

Appendix A: The golden rules of agentic coding for SaaS

1. **You are the architect; the agent is the builder.** Decide, then delegate.
2. **Plan before code, every time.** Fixing a plan is cheap; fixing code is expensive.
3. **Write it down.** `CLAUDE.md` / `AGENTS.md` and `/docs/` are the agent's memory. Without them, every session starts from zero.
4. **Small tasks, small diffs.** One feature per branch, one concern per commit.
5. **The agent must prove it works.** Tests, lint, typecheck, and output shown to you.
6. **Never let it edit a test to pass.** Bugs are fixed in code.
7. **Review with a fresh pair of eyes.** Use a new session or a reviewer subagent for security and code review.
8. **Server is the source of truth.** Validation, permissions, limits: always enforced on the backend.
9. **Every tenant query is scoped.** `organization_id` everywhere, tested.
10. **Read the diff.** If you don't understand a change, ask the agent to explain it before merging.

Appendix B: Pre-launch checklist

- ☐ Cross-tenant data access tested and blocked
- ☐ Auth via a proven provider; no custom password handling
- ☐ Every endpoint validates input and checks permissions on the server
- ☐ Payments driven by verified, idempotent webhooks
- ☐ CI blocks merges on failing tests; production deploy has rollback
- ☐ Critical flows covered by end-to-end tests
- ☐ Separate staging and production environments and databases
- ☐ Backups enabled and a restore actually tested
- ☐ Security review done; dependencies audited
- ☐ Rate limits on login, signup and expensive endpoints
- ☐ No tenant data cached at shared layers
- ☐ Errors reach your error tracker; logs are structured and redacted
- ☐ Uptime checks and alerts, each with a runbook
- ☐ Load tested at expected launch traffic
- ☐ Billing alerts set on every provider
- ☐ Terms of service and privacy policy published

AI wrote the code. You still hold all fifteen of these.